

Compte rendu

Étape 3 – Développement Web

1. Objectif

Pour cette étape, mon objectif était de créer l'interface utilisateur et le backend permettant de :

- ✓ Afficher les données de détection reçues en temps réel depuis le broker MQTT.
- ✓ Sauvegarder ces données dans une base de données pour une exploitation ultérieure.
- ✓ Permettre l'envoi de commandes à des actionneurs (porte, alarme) via MQTT.

Cette étape constitue la dernière partie du projet, reliant le système de détection (Étape 1) et la publication MQTT (Étape 2) à une interface pratique pour l'utilisateur.

2. Architecture Frontend / Backend

Backend

Le backend gère :

- ✓ La réception des messages MQTT de détection (iot/detection) et de commande (iot/command).
- ✓ Le stockage des données dans une base SQLite pour consultation ultérieure.
- ✓ L'envoi des commandes vers les actionneurs connectés.

Fichiers principaux backend :

1. mqtt_subscriber_sqlite.py → reçoit les messages de détection et les enregistre dans SQLite

```
# IMPORTATION DES LIBRAIRIES

import paho.mqtt.client as mqtt # Pour la communication MQTT

from datetime import datetime    # Pour gérer les timestamps

import json                     # Pour parser les messages JSON
```

```
import sqlite3          # Pour stocker les données localement dans SQLite

# CONFIGURATION MQTT

broker = "a0340c51dae9493c9ca67857a462c5fa.s1.eu.hivemq.cloud" # Adresse du broker
HiveMQ Cloud

port = 8883              # Port sécurisé TLS

username = "wiemchedly"  # Nom d'utilisateur

password = "Wiemlovemama123#" # Mot de passe

topic = "iot/detection"  # Topic MQTT auquel s'abonner

# CONFIGURATION SQLITE

# Création / ouverture de la base SQLite

conn = sqlite3.connect("iot.db", check_same_thread=False)

cursor = conn.cursor()

# Création de la table pour stocker les détections si elle n'existe pas

cursor.execute(""" CREATE TABLE IF NOT EXISTS detections (

    id INTEGER PRIMARY KEY AUTOINCREMENT, -- Identifiant unique automatique

    topic TEXT,                          -- Topic MQTT du message

    objet TEXT,                          -- Nom de l'objet détecté

    confiance REAL,                      -- Confiance de la détection

    received_at TEXT                     -- Date et heure de réception

) """)
```

```
conn.commit()

print("✓ Base SQLite prête")

# CALLBACKS MQTT

# Fonction appelée lors de la connexion au broker MQTT

def on_connect(client, userdata, flags, rc):

    if rc == 0:

        print("✓ Connecté au broker MQTT")

        client.subscribe(topic) # S'abonner au topic pour recevoir les messages

    else:

        print("✗ Erreur connexion MQTT :", rc)

# Fonction appelée à chaque réception d'un message sur le topic

def on_message(client, userdata, msg):

    try:

        payload = msg.payload.decode() # Décoder le message depuis bytes en string

        print("🔔 Message reçu dans subscriber :", payload)

        data = json.loads(payload) # Convertir le message JSON en dictionnaire Python

        timestamp = datetime.utcnow().isoformat() # Heure UTC au format ISO

        # Insertion des données reçues dans la table SQLite

        cursor.execute("INSERT INTO detections (topic, objet, confiance,
received_at) VALUES (?, ?, ?, ?)",
```

```

        (            msg.topic,

            data.get("objet"),    # Nom de l'objet

            data.get("confiance"), # Confiance associée

            timestamp            # Date et heure de réception        )    )

    conn.commit()            # Valider la transaction

    print("📁 Donnée enregistrée dans SQLite")

except Exception as e:

    print("❌ Erreur on_message :", e)

# CREATION DU CLIENT MQTT

client = mqtt.Client("Subscriber_SQLite") # Nom du client MQTT

client.username_pw_set(username, password) # Authentification MQTT

client.tls_set()                # Activation TLS pour sécuriser la connexion

client.tls_insecure_set(True)    # Ignorer la vérification du certificat (HiveMQ
Cloud)

client.on_connect = on_connect    # Définir la fonction de callback on_connect

client.on_message = on_message    # Définir la fonction de callback on_message

client.connect(broker, port)      # Connexion au broker MQTT

# Boucle infinie pour maintenir la connexion active et traiter les messages entrants

client.loop_forever()

```

2. mqtt_command_subscriber.py → reçoit les commandes et exécute les actions (porte/alarme)

```
# IMPORTATION DES LIBRAIRIES

import paho.mqtt.client as mqtt # Pour la communication MQTT
import json                     # Pour parser les messages JSON

# CONFIGURATION MQTT

broker = "a0340c51dae9493c9ca67857a462c5fa.s1.eu.hivemq.cloud" # Adresse du broker
HiveMQ Cloud
port = 8883 # Port sécurisé TLS
username = "wiemchedly" # Nom d'utilisateur MQTT
password = "Wiemlovemama123#" # Mot de passe MQTT
topic_cmd = "iot/command" # Topic sur lequel l'objet IoT attend
des commandes

# CALLBACK MQTT POUR LA CONNEXION

def on_connect(client, userdata, flags, rc):
    """
    Fonction appelée lorsque le client MQTT se connecte au broker.
    """
    if rc == 0:
        print("✓ Objet IoT connecté au broker")
        client.subscribe(topic_cmd) # S'abonner au topic des commandes
        print("✎ En attente de commandes IoT...")
    else:
        print("✗ Erreur connexion MQTT, code :", rc)

# CALLBACK MQTT POUR LES MESSAGES REÇUS

def on_message(client, userdata, msg):
    """
    Fonction appelée lorsqu'un message est reçu sur un topic auquel le client est abonné.
    """
    # Décoder le message JSON reçu
    data = json.loads(msg.payload.decode())

    # Récupérer les informations de l'objet et de l'action
    device = data.get("device")
    action = data.get("action")

    # GESTION DES COMMANDES POUR LA PORTE
    if device == "door":
```

```
if action == "OPEN":
    print("🚪 Porte OUVERTE")
elif action == "CLOSE":
    print("🔒 Porte FERMÉE")

# GESTION DES COMMANDES POUR L'ALARME
if device == "alarm":
    if action == "ON":
        print("🔊 Alarme ACTIVÉE")
    elif action == "OFF":
        print("🔇 Alarme DÉSACTIVÉE")

# CREATION DU CLIENT MQTT

client = mqtt.Client("IoT_Device")    # Nom du client MQTT
client.username_pw_set(username, password) # Authentification MQTT
client.tls_set()                      # Activation TLS pour sécuriser la connexion
client.tls_insecure_set(True)         # Ignorer la vérification du certificat (HiveMQ Cloud)

# Définir les callbacks
client.on_connect = on_connect
client.on_message = on_message

# CONNEXION AU BROKER MQTT

client.connect(broker, port)

# BOUCLE INFINIE POUR RECEVOIR LES COMMANDES

client.loop_forever() # Le client reste actif pour recevoir les messages en continu
```

Frontend

Le frontend est l'interface utilisateur développée avec **Streamlit** :

- ✓ Elle lit la base SQLite pour afficher les dernières détections et statistiques.
- ✓ Elle permet l'envoi de commandes vers le broker MQTT pour actionner les dispositifs IoT.

Fichier frontend :

- ✓ app.py → interface Streamlit pour visualisation et contrôle IoT

```
import streamlit as st
import sqlite3
import paho.mqtt.client as mqtt
import json
import pandas as pd

# CONFIG PAGE

st.set_page_config(
    page_title="Plateforme IoT – Détection Fruits",
    page_icon="🍏",
    layout="wide"
)

# STYLE CSS

st.markdown("""
<style>

/* TITRE PRINCIPAL */
.main-title {
    font-size: 36px;
    font-weight: 700;
    margin-bottom: 5px;
}

/* SOUS TITRES */
.section-title {
    font-size: 23px;
    font-weight: 600;
    margin-top: 10px;
    margin-bottom: 10px;
}

/* CARTES STATISTIQUES */
.stat-card {
    background-color: #f7f7f7;
    padding: 12px;
    border-radius: 10px;
    text-align: center;
    box-shadow: 0 2px 6px rgba(0,0,0,0.05);
}
```

```
.stat-card h3 {
    font-size: 16px;
    margin-bottom: 5px;
}

.stat-card h1 {
    font-size: 22px;
    margin: 0;
}

/* TABLEAU */
[data-testid="stDataFrame"] {
    max-width: 75%;
    margin: auto;
}

/* BOUTONS */
.stButton > button {
    padding: 8px 12px;
    font-size: 14px;
}

</style>
""", unsafe_allow_html=True)

# TITRE

st.markdown("<div class='main-title'>🍷🍏 Plateforme IoT – Détection de Fruits</div>",
unsafe_allow_html=True)
st.caption("Surveillance en temps réel & commandes IoT")
st.divider()

# MQTT (COMMANDES)

broker = "a0340c51dae9493c9ca67857a462c5fa.s1.eu.hivemq.cloud"
port = 8883
username = "wiemchedly"
password = "Wiemlovemama123#"
topic_cmd = "iot/command"

mqtt_client = mqtt.Client("Streamlit_CMD")
mqtt_client.username_pw_set(username, password)
mqtt_client.tls_set()
mqtt_client.tls_insecure_set(True)
mqtt_client.connect(broker, port)

# DATABASE
conn = sqlite3.connect("iot.db", check_same_thread=False)
```



```

cursor = conn.cursor()

cursor.execute("""
SELECT objet, confiance, received_at
FROM detections
ORDER BY received_at DESC
""")
rows = cursor.fetchall()

df = pd.DataFrame(rows, columns=["Objet", "Confiance", "Date & Heure"])

# STATISTIQUES
st.markdown("<div class='section-title'>📊 Statistiques</div>", unsafe_allow_html=True)

if not df.empty:
    stats = df["Objet"].value_counts()
    cols = st.columns(len(stats))

    for col, (obj, count) in zip(cols, stats.items()):
        with col:
            st.markdown(f"""
            <div class="stat-card">
            <h3>{obj}</h3>
            <h1>{count}</h1>
            <p>détections</p>
            </div>
            """, unsafe_allow_html=True)
    else:
        st.info("Aucune donnée disponible")

st.divider()

# DERNIÈRES DÉTECTIONS
st.markdown("<div class='section-title'>📁 Dernières détections</div>",
unsafe_allow_html=True)

if not df.empty:
    st.dataframe(df.head(10), use_container_width=False)
else:
    st.warning("Pas encore de détection")

st.divider()

# COMMANDES IoT
st.markdown("<div class='section-title'>📡 Commandes IoT</div>",
unsafe_allow_html=True)

col1, col2 = st.columns(2)

```

```
with col1:
    if st.button("🔓 Ouvrir la porte"):
        mqtt_client.publish(
            topic_cmd,
            json.dumps({"device": "door", "action": "OPEN"})
        )
        st.success("Commande envoyée : OUVRIR la porte")

    if st.button("🔒 Fermer la porte"):
        mqtt_client.publish(
            topic_cmd,
            json.dumps({"device": "door", "action": "CLOSE"})
        )
        st.success("Commande envoyée : FERMER la porte")

with col2:
    if st.button("🔔 Activer l'alarme"):
        mqtt_client.publish(
            topic_cmd,
            json.dumps({"device": "alarm", "action": "ON"})
        )
        st.success("Commande envoyée : ALARME ACTIVÉE")

    if st.button("🔕 Désactiver l'alarme"):
        mqtt_client.publish(
            topic_cmd,
            json.dumps({"device": "alarm", "action": "OFF"})
        )
        st.success("Commande envoyée : ALARME DÉSACTIVÉE")

st.divider()
st.caption("🔗 Projet IoT – YOLO • MQTT • SQLite • Streamlit")
```

3. Fonctionnalités implémentées

Backend

- ✓ Connexion sécurisée au broker MQTT (TLS).
- ✓ Subscription aux topics : `iot/detection` et `iot/command`.
- ✓ Stockage des données reçues dans SQLite :
 - Objet détecté
 - Confiance

- Date et heure de détection
- ✓ Réception et traitement des commandes envoyées par le frontend :
 - Ouvrir/Fermer la porte
 - Activer/Désactiver l'alarme

Frontend

- ✓ Visualisation en temps réel des détections :
 - Affichage des dernières 10 détections avec date/heure et niveau de confiance.
- ✓ Statistiques dynamiques des objets détectés : nombre de détections par type de fruit.
- ✓ Contrôle des actionneurs via boutons :
 -  Ouvrir/Fermer la porte
 -  Activer/Désactiver l'alarme

4. Fonctionnement

- ✓ Le backend s'abonne au topic `iot/detection` et reçoit les messages publiés par le publisher YOLO (Étape 2).
- ✓ Chaque message est stocké dans SQLite pour un usage futur.
- ✓ Le frontend Streamlit se connecte à la base SQLite pour afficher les données et statistiques.
- ✓ Les commandes envoyées via l'interface Streamlit sont publiées sur le topic `iot/command` et reçues par le backend pour actionner les dispositifs.

5. Exemple de visualisation et contrôle

- ✓ **Statistiques des fruits détectés** : affichage sous forme de cartes avec le nombre de détections par fruit.
- ✓ **Dernières détections** : tableau listant les objets détectés avec confiance et date/heure.
- ✓ **Commandes IoT** : boutons interactifs pour contrôler la porte et l'alarme.

Fusion des fichiers dans notre projet IoT

Au départ, notre projet avait trois fichiers :

1. `detect_and_publish.py` : détecte les fruits avec YOLO, les affiche avec un cadre vert et publie les résultats sur MQTT.
2. `mqtt_subscriber_sqlite.py` : reçoit les messages MQTT et les enregistre dans SQLite.
3. `mqtt_command_subscriber.py` : reçoit les commandes IoT et exécute les actions (porte/alarme).

Pour simplifier le projet, j'ai fusionné ces trois fonctionnalités dans un seul fichier `iot_backend.py`.

Cette fusion permet :

- d'exécuter tout le backend avec un seul fichier,
- de gérer la détection, l'affichage, l'enregistrement dans SQLite et les commandes IoT de manière centralisée,
- de faciliter la maintenance et la cohérence des données.

Le fichier `iot_backend.py`

```
from ultralytics import YOLO

import cv2

import paho.mqtt.client as mqtt

import json

import sqlite3

from datetime import datetime

# MQTT CONFIG

BROKER = "a0340c51dae9493c9ca67857a462c5fa.s1.eu.hivemq.cloud"
```

```
PORT = 8883

USERNAME = "wiemchedly"

PASSWORD = "Wiemlovemama123#"

TOPIC_DETECTION = "iot/detection"

TOPIC_COMMAND = "iot/command"

# DATABASE (SQLite)

conn = sqlite3.connect("iot.db", check_same_thread=False)

cursor = conn.cursor()

cursor.execute("""

CREATE TABLE IF NOT EXISTS detections (

    id INTEGER PRIMARY KEY AUTOINCREMENT,

    objet TEXT,

    confiance REAL,

    received_at TEXT

)

""")

conn.commit()

print("✓ Backend IoT prêt")

# MQTT CALLBACKS

def on_connect(client, userdata, flags, rc):
```

```
if rc == 0:

    print("✔ Connecté au broker MQTT")

    client.subscribe(TOPIC_COMMAND)

else:

    print("✗ Erreur connexion MQTT")

def on_message(client, userdata, msg):

    data = json.loads(msg.payload.decode())

    if msg.topic == TOPIC_COMMAND:

        device = data.get("device")

        action = data.get("action")

        if device == "door":

            print("🚪 Porte :", action)

        elif device == "alarm":

            print("🚨 Alarme :", action)

# MQTT CLIENT

mqtt_client = mqtt.Client("IoT_Backend")

mqtt_client.username_pw_set(USERNAME, PASSWORD)

mqtt_client.tls_set()

mqtt_client.tls_insecure_set(True)
```

```
mqtt_client.on_connect = on_connect

mqtt_client.on_message = on_message

mqtt_client.connect(BROKER, PORT)

mqtt_client.loop_start()

# YOLO MODEL

model = YOLO("C:/Users/ASUS/Desktop/Projet_IOT_Wiem/yolov8s.pt")

# Classes fruits COCO

# 46 = banana | 47 = apple | 49 = orange

FRUIT_CLASSES = [46, 47, 49]

# WEBCAM

cap = cv2.VideoCapture(0)

print("📷 Webcam active (appuyez sur Q pour quitter)")

while True:

    ret, frame = cap.read()

    if not ret:

        break

    results = model(frame, stream=True, verbose=False)

    for r in results:

        for box in r.bboxes:

            cls_id = int(box.cls[0])
```

```
conf = float(box.conf[0])

if cls_id in FRUIT_CLASSES and conf > 0.6:

    name = model.names[cls_id]

    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

    # DRAW BOUNDING BOX

    x1, y1, x2, y2 = map(int, box.xyxy[0])

    cv2.rectangle(

        frame,

        (x1, y1),

        (x2, y2),

        (0, 255, 0),

        2

    )

    cv2.putText(

        frame,

        f"{name} {conf:.2f}",

        (x1, y1 - 10),

        cv2.FONT_HERSHEY_SIMPLEX,

        0.8,

        (0, 255, 0),
```



```
2

)

# MQTT PUBLISH

message = {

    "objet": name,

    "confiance": round(conf, 3),

    "heure": timestamp

}

mqtt_client.publish(TOPIC_DETECTION, json.dumps(message))

# DATABASE INSERT

cursor.execute(

    "INSERT INTO detections (objet, confiance, received_at) VALUES (?, ?, ?)",

    (name, round(conf, 3), timestamp)

)

conn.commit()

print("📡 Détection envoyée & enregistrée :", message)

cv2.imshow("Detection IoT", frame)

if cv2.waitKey(1) & 0xFF == ord("q"):

    break
```

```
# CLEAN EXIT

cap.release()

cv2.destroyAllWindows()

mqtt_client.loop_stop()

mqtt_client.disconnect()

conn.close()

print("● Backend IoT arrêté")
```

Le fichier forontend : app.py

```
import streamlit as st

import sqlite3

import paho.mqtt.client as mqtt

import json

import pandas as pd

# CONFIG PAGE

st.set_page_config(

    page_title="Plateforme IoT – Détection Fruits",

    page_icon="🍏",

    layout="wide"
```

```
)

# STYLE CSS

st.markdown("""

<style>

/* TITRE PRINCIPAL */

.main-title {

    font-size: 36px;

    font-weight: 700;

    margin-bottom: 5px;

}

/* SOUS TITRES */

.section-title {

    font-size: 23px;

    font-weight: 600;

    margin-top: 10px;

    margin-bottom: 10px;

}

/* CARTES STATISTIQUES */

.stat-card {

    background-color: #f7f7f7;
```

```
padding: 12px;

border-radius: 10px;

text-align: center;

box-shadow: 0 2px 6px rgba(0,0,0,0.05);

}

.stat-card h3 {

    font-size: 16px;

    margin-bottom: 5px;

}

.stat-card h1 {

    font-size: 22px;

    margin: 0;

}

/* TABLEAU */

[data-testid="stDataFrame"] {

    max-width: 75%;

    margin: auto;

}
```

```
/* BOUTONS */

.stButton > button {

    padding: 8px 12px;

    font-size: 14px;

}

</style>

"", unsafe_allow_html=True)

# TITRE

st.markdown("<div class='main-title'>🍷🍏 Plateforme IoT – Détection de Fruits</div>",
unsafe_allow_html=True)

st.caption("Surveillance en temps réel & commandes IoT")

st.divider()

# MQTT (COMMANDES)

broker = "a0340c51dae9493c9ca67857a462c5fa.s1.eu.hivemq.cloud"

port = 8883

username = "wiemchedly"

password = "Wiemlovemama123#"

topic_cmd = "iot/command"

mqtt_client = mqtt.Client("Streamlit_CMD")

mqtt_client.username_pw_set(username, password)
```

```
mqtt_client.tls_set()

mqtt_client.tls_insecure_set(True)

mqtt_client.connect(broker, port)

# DATABASE

conn = sqlite3.connect("iot.db", check_same_thread=False)

cursor = conn.cursor()

cursor.execute("""

SELECT objet, confiance, received_at

FROM detections

ORDER BY received_at DESC

""")

rows = cursor.fetchall()

df = pd.DataFrame(rows, columns=["Objet", "Confiance", "Date & Heure"])

# STATISTIQUES

st.markdown("<div class='section-title'>📊 Statistiques</div>", unsafe_allow_html=True)

if not df.empty:

    stats = df["Objet"].value_counts()

    cols = st.columns(len(stats))

    for col, (obj, count) in zip(cols, stats.items()):

        with col:
```

```
st.markdown(f"""

    <div class="stat-card">

        <h3>{obj}</h3>

        <h1>{count}</h1>

        <p>détections</p>

    </div>

    """, unsafe_allow_html=True)

else:

    st.info("Aucune donnée disponible")

st.divider()

# DERNIÈRES DÉTECTIONS

st.markdown("<div class='section-title'>📁 Dernières détections</div>",
unsafe_allow_html=True)

if not df.empty:

    st.dataframe(df.head(10), use_container_width=False)

else:

    st.warning("Pas encore de détection")

st.divider()

# COMMANDES IoT
```

```
st.markdown("<div class='section-title'>🔑 □ Commandes IoT</div>",
unsafe_allow_html=True)

col1, col2 = st.columns(2)

with col1:

    if st.button("🔓 Ouvrir la porte"):

        mqtt_client.publish(

            topic_cmd,

            json.dumps({"device": "door", "action": "OPEN"})

        )

        st.success("Commande envoyée : OUVRIR la porte")


    if st.button("🔒 Fermer la porte"):

        mqtt_client.publish(

            topic_cmd,

            json.dumps({"device": "door", "action": "CLOSE"})

        )

        st.success("Commande envoyée : FERMER la porte")

with col2:

    if st.button("🚨 Activer l'alarme"):

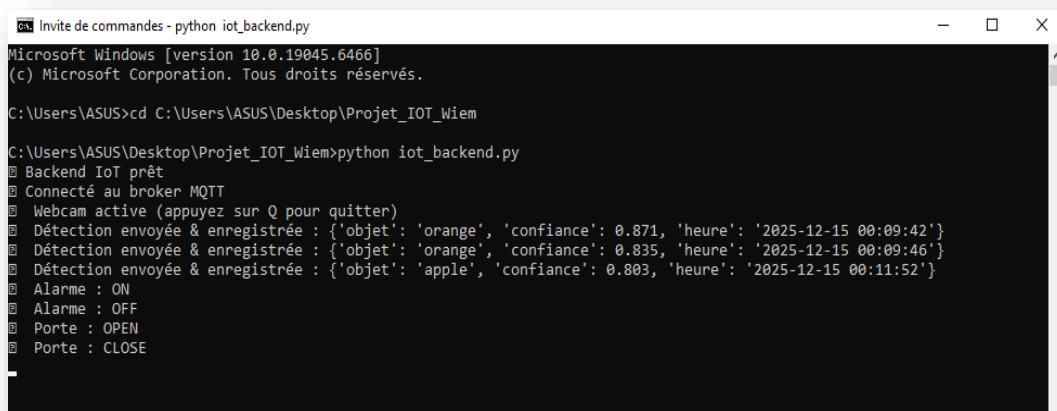
        mqtt_client.publish(
```



```
topic_cmd,  
  
    json.dumps({"device": "alarm", "action": "ON"})  
  
    )  
  
    st.success("Commande envoyée : ALARME ACTIVÉE")  
  
if st.button("🔔 Désactiver l'alarme"):  
  
    mqtt_client.publish(  
  
        topic_cmd,  
  
        json.dumps({"device": "alarm", "action": "OFF"})    )  
  
    st.success("Commande envoyée : ALARME DÉSACTIVÉE")  
  
st.divider()  
  
st.caption("🔧 Projet IoT – YOLO • MQTT • SQLite • Streamlit")
```

Terminal 1

Python `iot_python.py`



```
Invite de commandes - python iot_backend.py
Microsoft Windows [version 10.0.19045.6466]
(c) Microsoft Corporation. Tous droits réservés.

C:\Users\ASUS>cd C:\Users\ASUS\Desktop\Projet_IOT_Wiem

C:\Users\ASUS\Desktop\Projet_IOT_Wiem>python iot_backend.py
Backend IoT prêt
Connecté au broker MQTT
Webcam active (appuyez sur Q pour quitter)
Détection envoyée & enregistrée : {'objet': 'orange', 'confiance': 0.871, 'heure': '2025-12-15 00:09:42'}
Détection envoyée & enregistrée : {'objet': 'orange', 'confiance': 0.835, 'heure': '2025-12-15 00:09:46'}
Détection envoyée & enregistrée : {'objet': 'apple', 'confiance': 0.803, 'heure': '2025-12-15 00:11:52'}
Alarme : ON
Alarme : OFF
Porte : OPEN
Porte : CLOSE
```

Terminal 2

Streamlit run app.py

```
Invite de commandes - streamlit run app.py
Microsoft Windows [version 10.0.19045.6466]
(c) Microsoft Corporation. Tous droits réservés.

C:\Users\ASUS>cd C:\Users\ASUS\Desktop\Projet_IOT_Wiem

C:\Users\ASUS\Desktop\Projet_IOT_Wiem>streamlit run app.py

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.0.122:8501
```



Dernières détections

	Objet	Confiance	Date & Heure
0	apple	0.803	2025-12-15 00:11:52
1	orange	0.835	2025-12-15 00:09:46
2	orange	0.871	2025-12-15 00:09:42

Commandes IoT

Ouvrir la porte Activer l'alarme

Fermer la porte Désactiver l'alarme

Commande envoyée : FERMER la porte

Technologies utilisées – (Frontend / Application Web)

J'ai développé une interface web pour afficher les détections et envoyer des commandes IoT.

- ✓ **Python** : Langage principal pour la logique de détection, le traitement des données et la communication MQTT.
- ✓ **Streamlit** : Framework Python pour créer l'interface web ; il génère automatiquement du HTML, CSS et JavaScript.
- ✓ **CSS** : Personnalisation du style de l'interface (titres, boutons, tableaux) directement dans `app.py`.
- ✓ **MQTT (Paho MQTT)** : Pour la communication entre l'objet IoT, le backend et l'interface.
- ✓ **SQLite** : Base de données locale pour stocker les détections en temps réel.
- ✓ **OpenCV** : Capture vidéo depuis la webcam et traitement des images.
- ✓ **YOLO (Ultralytics)** : Détection d'objets (fruits) à partir du flux vidéo.

6. Conclusion

Grâce à cette étape :

- ✓ J'ai créé une **interface utilisateur fonctionnelle** pour visualiser les détections et envoyer des commandes.
- ✓ Le **backend** permet de centraliser les informations et de communiquer avec les actionneurs.
- ✓ Cette étape finalise le projet en liant la détection d'objets à une **plateforme IoT complète et interactive**.